

Supplemental: Formalized Lean 4 Proofs Local Softmax and Global Weights in Non-Boolean Event Structures

Karl Svozil 

May 31, 2026

Overview

This document presents the complete, verbatim source code for the formal verification accompanying the main manuscript. The executable project workspace is permanently archived on Zenodo at <https://doi.org/10.5281/zenodo.20479152> and can be cross-referenced with the active development repository.

This formalized supplement provides a complete machine-checked proof certificate for all finite claims concerning context-wise gluing, generalized-softmax coordinates, and the boundary bounds of the pentagon half-weight. The entire codebase has been fully verified using the Lean 4 interactive theorem prover (v4.30.0) and its standard mathematical library, mathlib (v4.30.0). The project compiles successfully via the standard `lake build` workflow; the underlying source text contains no logical gaps, placeholders (`sorry`, `admit`), or additional unvetted foundational assumptions (`axiom`).

The formalization separates three layers:

- First, Section 2 (`Core.lean`) defines finite event structures, admissible weights, positive coordinates, and the representation theorem for scaled link-function preimages.
- Second, Section 3 (`Gluing.lean`) formalizes local context-wise distributions, single-valuedness, the normalizer-ratio calculation, and the theorem that single-valued local distributions glue to a global admissible weight.
- Third, Section 4 (`Pentagon.lean`) formalizes the pentagon example: the half-weight is admissible, every two-valued state and every finite classical mixture obeys the cyclic bound 2, the half-weight has cyclic sum $5/2$, and $5/2 > \sqrt{5}$. The same file also verifies the positive softmax-coordinate path and its boundary endpoint at the half-weight.

Contents

1	Project Configuration files	3
2	Core representation theorem	4
3	Gluing Logic	8
4	Pentagon Bounds Verification	11

1 Project Configuration files

lean-toolchain

```
leanprover/lean4:v4.30.0
```

lakefile.lean

```
1 import Lake
2 open Lake DSL
3
4 package SoftmaxVerification where
5
6 require mathlib from git
7   "https://github.com/leanprover-community/mathlib4.git" @ "v4.30.0"
8
9 @[default_target]
10 lean_lib SoftmaxVerification where
```

SoftmaxVerification.lean

```
1 import SoftmaxVerification.Core
2 import SoftmaxVerification.Gluing
3 import SoftmaxVerification.Pentagon
```

2 Core representation theorem

Core.lean

```
1 import Mathlib
2
3 open scoped BigOperators
4
5 namespace SoftmaxVerification
6
7 /--
8 A finite event structure in the sense used in the softmax paper.
9
10 `atoms` is the ambient finite set of atoms. Each context is itself a finite
11 set of atoms, and `context_subset` records that contexts do not contain
12 anything outside the ambient atom set.
13 -/
14 structure EventStructure (Atom : Type*) [DecidableEq Atom] where
15   atoms : Finset Atom
16   contexts : Finset (Finset Atom)
17   context_subset : forall C, C ∈ contexts -> C ⊆ atoms
18
19 /--
20 An admissible weight on an event structure: nonnegative on atoms and
21 normalized to one on every context.
22 -/
23 structure IsWeight {Atom : Type*} [DecidableEq Atom]
24   (L : EventStructure Atom) (p : Atom -> Real) : Prop where
25   nonneg : forall a, a ∈ L.atoms -> 0 ≤ p a
26   normalized : forall C, C ∈ L.contexts -> Finset.sum C p = 1
27
28 /-- Strict positivity on all atoms of the event structure. -/
29 def StrictlyPositive {Atom : Type*} [DecidableEq Atom]
30   (L : EventStructure Atom) (p : Atom -> Real) : Prop :=
31   forall a, a ∈ L.atoms -> 0 < p a
32
33 theorem strictlyPositive_nonneg {Atom : Type*} [DecidableEq Atom]
34   {L : EventStructure Atom} {p : Atom -> Real}
35   (hpos : StrictlyPositive L p) :
36   forall a, a ∈ L.atoms -> 0 ≤ p a := by
37   intro a ha
38   exact le_of_lt (hpos a ha)
39
40 /-- The normalizing denominator for positive coordinates on a context. -/
41 def denom {Atom : Type*} (q : Atom -> Real) (C : Finset Atom) : Real :=
42   Finset.sum C q
43
44 /-- Context-wise normalization of coordinates. -/
45 noncomputable def normalizedCoordinate {Atom : Type*}
```

```

46   (q : Atom -> Real) (C : Finset Atom) (a : Atom) : Real :=
47   q a / denom q C
48
49 /--
50 `q` represents the admissible weight `p` by normalized coordinates if the
51 coordinates are positive on atoms and context-wise normalization recovers `p`.
52 -/
53 def Represents {Atom : Type*} [DecidableEq Atom]
54   (L : EventStructure Atom) (p q : Atom -> Real) : Prop :=
55   (forall a, a ∈ L.atoms -> 0 < q a) ∧
56   forall C, C ∈ L.contexts -> forall a, a ∈ C ->
57     normalizedCoordinate q C a = p a
58
59 /--
60 Coordinate form of the paper's representation theorem.
61
62 Every strictly positive admissible weight represents itself: take the
63 unnormalized coordinate of an atom to be its weight.
64 -/
65 theorem positive_weight_represents_itself {Atom : Type*} [DecidableEq Atom]
66   {L : EventStructure Atom} {p : Atom -> Real}
67   (hp : IsWeight L p) (hpos : StrictlyPositive L p) :
68   Represents L p p := by
69   constructor
70   · exact hpos
71   · intro C hC a ha
72     unfold normalizedCoordinate denom
73     rw [hp.normalized C hC]
74     simp
75
76 /--
77 The same representation after multiplying all coordinates by a positive
78 constant. This is the basic gauge freedom of normalized coordinates.
79 -/
80 theorem positive_weight_scaled_coordinates {Atom : Type*} [DecidableEq Atom]
81   {L : EventStructure Atom} {p : Atom -> Real}
82   (hp : IsWeight L p) (hpos : StrictlyPositive L p)
83   {alpha : Real} (halpha : 0 < alpha) :
84   Represents L p (fun a => alpha * p a) := by
85   constructor
86   · intro a ha
87     exact mul_pos halpha (hpos a ha)
88   · intro C hC a ha
89     unfold normalizedCoordinate denom
90     have hsum : Finset.sum C (fun b => alpha * p b) = alpha := by
91       calc
92         Finset.sum C (fun b => alpha * p b) = alpha * Finset.sum C p := by
93           rw [Finset.mul_sum]
94         _ = alpha * 1 := by
95           rw [hp.normalized C hC]

```

```

96     _ = alpha := by
97       ring
98       rw [hsum]
99       field_simp [ne_of_gt halpha]
100
101 /--
102 Link-function version of the representation theorem.
103
104 Instead of formalizing an inverse function immediately, this theorem states the
105 exact range condition used in the paper: if every scaled value `alpha * p a`
106 has a preimage under the link `g`, then choosing those preimages as scores
107 gives the desired generalized-softmax representation.
108 -/
109 theorem link_representation_of_scaled_range
110   {Atom Score : Type*} [DecidableEq Atom] [Nonempty Score]
111   {L : EventStructure Atom} {p : Atom -> Real}
112   (g : Score -> Real)
113   (hp : IsWeight L p)
114   {alpha : Real} (halpha : 0 < alpha)
115   (hpre : forall a, a ∈ L.atoms -> exists u : Score, g u = alpha * p a) :
116   exists u : Atom -> Score,
117     forall C, C ∈ L.contexts -> forall a, a ∈ C ->
118       normalizedCoordinate (fun b => g (u b)) C a = p a := by
119   classical
120   let u : Atom -> Score := fun a =>
121     if ha : a ∈ L.atoms then
122       Classical.choose (hpre a ha)
123     else
124       Classical.choice (inferInstance : Nonempty Score)
125   refine {u, ?}_
126   intro C hC a ha
127   have hCsub : C ⊆ L.atoms := L.context_subset C hC
128   have hu : forall b, b ∈ C -> g (u b) = alpha * p b := by
129     intro b hb
130     have hb_atoms : b ∈ L.atoms := hCsub hb
131     dsimp [u]
132     rw [dif_pos hb_atoms]
133     exact Classical.choose_spec (hpre b hb_atoms)
134   unfold normalizedCoordinate denom
135   have hsum : Finset.sum C (fun b => g (u b)) = alpha := by
136     calc
137       Finset.sum C (fun b => g (u b)) = Finset.sum C (fun b => alpha * p b) :=
138         by
139           apply Finset.sum_congr rfl
140           intro b hb
141           exact hu b hb
142       _ = alpha * Finset.sum C p := by
143         rw [Finset.mul_sum]
144       _ = alpha * 1 := by
145         rw [hp.normalized C hC]

```

```
145   _ = alpha := by
146     ring
147     change g (u a) / Finset.sum C (fun b => g (u b)) = p a
148     rw [hu a ha, hsum]
149     field_simp [ne_of_gt halpha]
150
151 end SoftmaxVerification
```

3 Gluing Logic

Gluing.lean

```
1 import SoftmaxVerification.Core
2
3 open scoped BigOperators
4
5 namespace SoftmaxVerification
6
7 /-- A context-indexed family of local probabilities is nonnegative on
8 the atoms of each context. -/
9 def LocalNonnegative {Atom : Type*} [DecidableEq Atom]
10   (L : EventStructure Atom) (P : Finset Atom -> Atom -> Real) : Prop :=
11   forall C, C ∈ L.contexts -> forall a, a ∈ C -> 0 <= P C a
12
13 /-- A context-indexed family of local probabilities is normalized in
14 each context. -/
15 def LocalNormalized {Atom : Type*} [DecidableEq Atom]
16   (L : EventStructure Atom) (P : Finset Atom -> Atom -> Real) : Prop :=
17   forall C, C ∈ L.contexts -> Finset.sum C (P C) = 1
18
19 /-- Single-valuedness/no-disturbance for context-indexed local
20 probabilities. Shared atoms receive the same value in all contexts in
21 which they occur. -/
22 def SingleValuedOn {Atom : Type*} [DecidableEq Atom]
23   (L : EventStructure Atom) (P : Finset Atom -> Atom -> Real) : Prop :=
24   forall C, C ∈ L.contexts -> forall D, D ∈ L.contexts -> forall a,
25     a ∈ C -> a ∈ D -> P C a = P D a
26
27 /-- Local normalizing denominator for freely context-dependent
28 positive coordinates. -/
29 def localDenom {Atom : Type*}
30   (q : Finset Atom -> Atom -> Real) (C : Finset Atom) : Real :=
31   Finset.sum C (q C)
32
33 /-- Local softmax normalization for freely context-dependent positive
34 coordinates. -/
35 noncomputable def localCoordinate {Atom : Type*}
36   (q : Finset Atom -> Atom -> Real) (C : Finset Atom) (a : Atom) : Real :=
37   q C a / localDenom q C
38
39 /-- The paper's gluing condition, stated as single-valuedness of the
40 locally normalized coordinates. -/
41 theorem gluing_condition_for_free_scores {Atom : Type*} [DecidableEq Atom]
42   {L : EventStructure Atom} {q : Finset Atom -> Atom -> Real} :
43   SingleValuedOn L (localCoordinate q) <=>
44     forall C, C ∈ L.contexts -> forall D, D ∈ L.contexts -> forall a,
45       a ∈ C -> a ∈ D ->
```



```

46     localCoordinate q C a = localCoordinate q D a := by
47     rfl
48
49     /-- Algebraic form of the normalizer-ratio condition for one shared
50     atom. This is the scalar calculation behind Eq. (4.3) in the paper. -/
51     theorem normalized_eq_iff_ratio
52       {qC qD ZC ZD : Real}
53       (hqD : qD ≠ 0) (hZC : ZC ≠ 0) (hZD : ZD ≠ 0) :
54       qC / ZC = qD / ZD <=> ZC / ZD = qC / qD := by
55     constructor
56     · intro h
57       field_simp [hZC, hZD] at h
58       field_simp [hZD, hqD]
59       nlinarith
60     · intro h
61       field_simp [hZD, hqD] at h
62       field_simp [hZC, hZD]
63       nlinarith
64
65     /-- A noncomputable glued global weight: choose one context containing
66     the atom, if such a context exists, and otherwise use zero. The choice
67     is harmless under `SingleValuedOn`. -/
68     noncomputable def gluedWeight {Atom : Type*} [DecidableEq Atom]
69       (L : EventStructure Atom) (P : Finset Atom -> Atom -> Real) :
70       Atom -> Real :=
71     fun a =>
72       if h : exists C, C ∈ L.contexts ∧ a ∈ C then
73         P (Classical.choose h) a
74       else
75         0
76
77     theorem gluedWeight_eq_of_mem {Atom : Type*} [DecidableEq Atom]
78       {L : EventStructure Atom} {P : Finset Atom -> Atom -> Real}
79       (hsingle : SingleValuedOn L P)
80       {C : Finset Atom} (hC : C ∈ L.contexts) {a : Atom} (ha : a ∈ C) :
81       gluedWeight L P a = P C a := by
82     classical
83     unfold gluedWeight
84     have hex : exists D, D ∈ L.contexts ∧ a ∈ D := {C, hC, }ha
85     rw [dif_pos hex]
86     have hchoose := Classical.choose_spec hex
87     exact hsingle (Classical.choose hex) hchoose.1 C hC a hchoose.2 ha
88
89     /-- Proposition 5.1 in finite form: every single-valued family of
90     locally normalized distributions defines an admissible global weight. -/
91     theorem local_distributions_glue_to_weight {Atom : Type*} [DecidableEq Atom]
92       {L : EventStructure Atom} {P : Finset Atom -> Atom -> Real}
93       (hnonneg : LocalNonnegative L P)
94       (hnorm : LocalNormalized L P)
95       (hsingle : SingleValuedOn L P) :

```

```

96   exists p : Atom -> Real,
97     IsWeight L p ∧
98     forall C, C ∈ L.contexts -> forall a, a ∈ C -> p a = P C a := by
99   classical
100  refine (gluedWeight L P, ?_, ?)_
101  · constructor
102  · intro a ha
103    unfold gluedWeight
104    by_cases hex : exists C, C ∈ L.contexts ∧ a ∈ C
105    · rw [dif_pos hex]
106      have hchoose := Classical.choose_spec hex
107      exact hnonneg (Classical.choose hex) hchoose.1 a hchoose.2
108    · rw [dif_neg hex]
109  · intro C hC
110    calc
111      Finset.sum C (gluedWeight L P) = Finset.sum C (P C) := by
112        apply Finset.sum_congr rfl
113        intro a ha
114        exact gluedWeight_eq_of_mem hsingle hC ha
115      _ = 1 := hnorm C hC
116  · intro C hC a ha
117    exact gluedWeight_eq_of_mem hsingle hC ha
118
119 end SoftmaxVerification

```

4 Pentagon Bounds Verification

Pentagon.lean

```
1 import SoftmaxVerification.Gluing
2
3 open scoped BigOperators
4
5 namespace SoftmaxVerification
6
7 /-- The ten atoms of the pentagon logic. -/
8 inductive PentagonAtom where
9   | a0 | a1 | a2 | a3 | a4
10  | x0 | x1 | x2 | x3 | x4
11  deriving DecidableEq, Fintype
12
13 open PentagonAtom
14
15 def pentagonC0 : Finset PentagonAtom := {a0, a1, x0}
16 def pentagonC1 : Finset PentagonAtom := {a1, a2, x1}
17 def pentagonC2 : Finset PentagonAtom := {a2, a3, x2}
18 def pentagonC3 : Finset PentagonAtom := {a3, a4, x3}
19 def pentagonC4 : Finset PentagonAtom := {a4, a0, x4}
20
21 def pentagonContexts : Finset (Finset PentagonAtom) :=
22   {pentagonC0, pentagonC1, pentagonC2, pentagonC3, pentagonC4}
23
24 /-- The pentagon/pentagram event structure from the paper. -/
25 def pentagon : EventStructure PentagonAtom where
26   atoms := Finset.univ
27   contexts := pentagonContexts
28   context_subset := by
29     intro C hC a ha
30     simp
31
32 /-- The cyclic atoms `a_i`. -/
33 def pentagonCyclicAtoms : Finset PentagonAtom := {a0, a1, a2, a3, a4}
34
35 def pentagonCyclicSum (p : PentagonAtom -> Real) : Real :=
36   Finset.sum pentagonCyclicAtoms p
37
38 /-- The pentagon half-weight: cyclic atoms have weight `1/2`, midpoint
39 atoms have weight `0`. -/
40 noncomputable def pentagonHalfWeight : PentagonAtom -> Real
41   | a0 | a1 | a2 | a3 | a4 => 1 / 2
42   | x0 | x1 | x2 | x3 | x4 => 0
43
44 theorem pentagon_halfWeight_isWeight : IsWeight pentagon pentagonHalfWeight :=
45   by
```

```

45 constructor
46   · intro a ha
47     cases a <=> norm_num [pentagonHalfWeight]
48   · intro C hC
49     simp [pentagon, pentagonContexts] at hC
50     rcases hC with hC | hC | hC | hC | hC
51     · subst C
52       simp [pentagonC0, pentagonHalfWeight]
53       norm_num
54     · subst C
55       simp [pentagonC1, pentagonHalfWeight]
56       norm_num
57     · subst C
58       simp [pentagonC2, pentagonHalfWeight]
59       norm_num
60     · subst C
61       simp [pentagonC3, pentagonHalfWeight]
62       norm_num
63     · subst C
64       simp [pentagonC4, pentagonHalfWeight]
65       norm_num
66
67 theorem pentagon_halfWeight_cyclicSum :
68   pentagonCyclicSum pentagonHalfWeight = 5 / 2 := by
69   simp [pentagonCyclicSum, pentagonCyclicAtoms, pentagonHalfWeight]
70   norm_num
71
72 /-- A two-valued admissible weight. -/
73 def IsTwoValued {Atom : Type*} [DecidableEq Atom]
74   (L : EventStructure Atom) (v : Atom -> Real) : Prop :=
75   IsWeight L v ∧ forall a, a ∈ L.atoms -> v a = 0 ∨ v a = 1
76
77 private theorem pentagon_adjacent_sum_le_one
78   {v : PentagonAtom -> Real} (hv : IsTwoValued pentagon v) :
79   v a0 + v a1 <= 1 ∧
80   v a1 + v a2 <= 1 ∧
81   v a2 + v a3 <= 1 ∧
82   v a3 + v a4 <= 1 ∧
83   v a4 + v a0 <= 1 := by
84   have hx0 : 0 <= v x0 := hv.1.nonneg x0 (by simp [pentagon])
85   have hx1 : 0 <= v x1 := hv.1.nonneg x1 (by simp [pentagon])
86   have hx2 : 0 <= v x2 := hv.1.nonneg x2 (by simp [pentagon])
87   have hx3 : 0 <= v x3 := hv.1.nonneg x3 (by simp [pentagon])
88   have hx4 : 0 <= v x4 := hv.1.nonneg x4 (by simp [pentagon])
89   have h0 := hv.1.normalized pentagonC0 (by simp [pentagon, pentagonContexts])
90   have h1 := hv.1.normalized pentagonC1 (by simp [pentagon, pentagonContexts])
91   have h2 := hv.1.normalized pentagonC2 (by simp [pentagon, pentagonContexts])
92   have h3 := hv.1.normalized pentagonC3 (by simp [pentagon, pentagonContexts])
93   have h4 := hv.1.normalized pentagonC4 (by simp [pentagon, pentagonContexts])
94   simp [pentagonC0] at h0

```

```

95 simp [pentagonC1] at h1
96 simp [pentagonC2] at h2
97 simp [pentagonC3] at h3
98 simp [pentagonC4] at h4
99 constructor
100 · nlinarith
101 constructor
102 · nlinarith
103 constructor
104 · nlinarith
105 constructor
106 · nlinarith
107 · nlinarith
108
109 /-- Every two-valued pentagon weight assigns value one to at most two
110 cyclic atoms. -/
111 theorem pentagon_twoValued_cyclicSum_le_two
112   {v : PentagonAtom -> Real} (hv : IsTwoValued pentagon v) :
113   pentagonCyclicSum v <= 2 := by
114   have hadj := pentagon_adjacent_sum_le_one hv
115   rcases hadj with (h01, h12, h23, h34, )h40
116   have ha0 := hv.2 a0 (by simp [pentagon])
117   have ha1 := hv.2 a1 (by simp [pentagon])
118   have ha2 := hv.2 a2 (by simp [pentagon])
119   have ha3 := hv.2 a3 (by simp [pentagon])
120   have ha4 := hv.2 a4 (by simp [pentagon])
121   rcases ha0 with ha0 | ha0 <=>
122   rcases ha1 with ha1 | ha1 <=>
123   rcases ha2 with ha2 | ha2 <=>
124   rcases ha3 with ha3 | ha3 <=>
125   rcases ha4 with ha4 | ha4 <=>
126   simp [pentagonCyclicSum, pentagonCyclicAtoms, ha0, ha1, ha2, ha3, ha4] <=>
127   nlinarith
128
129 /-- A finite classical mixture of two-valued pentagon weights obeys the
130 same cyclic inequality. -/
131 theorem pentagon_classical_mixture_cyclicSum_le_two
132   {ι : Type*} [Fintype ι]
133   {lambda : ι -> Real} {v : ι -> PentagonAtom -> Real}
134   {p : PentagonAtom -> Real}
135   (hlambda_nonneg : forall i, 0 <= lambda i)
136   (hlambda_sum : Finset.sum Finset.univ lambda = 1)
137   (hv : forall i, IsTwoValued pentagon (v i))
138   (hp : forall a, p a = Finset.sum Finset.univ (fun i => lambda i * v i a)) :
139   pentagonCyclicSum p <= 2 := by
140   have hsum :
141     pentagonCyclicSum p =
142       Finset.sum Finset.univ (fun i => lambda i * pentagonCyclicSum (v i)) :=
143       by
144         unfold pentagonCyclicSum

```

```

144   calc
145     Finset.sum pentagonCyclicAtoms p =
146       Finset.sum pentagonCyclicAtoms
147         (fun a => Finset.sum Finset.univ (fun i => lambda i * v i a)) := by
148         apply Finset.sum_congr rfl
149         intro a ha
150         exact hp a
151   _ = Finset.sum Finset.univ
152     (fun i => Finset.sum pentagonCyclicAtoms (fun a => lambda i * v i a
153       )) := by
154     rw [Finset.sum_comm]
155     _ = Finset.sum Finset.univ (fun i => lambda i * Finset.sum
156       pentagonCyclicAtoms (v i)) := by
157     apply Finset.sum_congr rfl
158     intro i hi
159     rw [Finset.mul_sum]
160   rw [hsum]
161   calc
162     Finset.sum Finset.univ (fun i => lambda i * pentagonCyclicSum (v i))
163     <= Finset.sum Finset.univ (fun i => lambda i * 2) := by
164     apply Finset.sum_le_sum
165     intro i hi
166     exact mul_le_mul_of_nonneg_left (pentagon_twoValued_cyclicSum_le_two (h
167       v i))
168     (hlambda_nonneg i)
169   _ = 2 := by
170   rw <[ Finset.sum_mul]
171   rw [hlambda_sum]
172   ring
173
174   theorem pentagon_halfWeight_exceeds_classical_bound :
175     2 < pentagonCyclicSum pentagonHalfWeight := by
176     rw [pentagon_halfWeight_cyclicSum]
177     norm_num
178
179   theorem pentagon_halfWeight_exceeds_kcbs_value :
180     Real.sqrt 5 < pentagonCyclicSum pentagonHalfWeight := by
181     rw [pentagon_halfWeight_cyclicSum]
182     rw [Real.sqrt_lt' (by norm_num : 0 < (5 : Real) / 2)]
183     norm_num
184
185   /- The positive generalized-softmax path used in the paper, written in
186   terms of the coordinate ratio `r`. -/
187   noncomputable def pentagonPathWeight (r : Real) : PentagonAtom -> Real
188   | a0 | a1 | a2 | a3 | a4 => 1 / (2 + r)
189   | x0 | x1 | x2 | x3 | x4 => r / (2 + r)
190
191   theorem pentagon_pathWeight_isWeight {r : Real} (hr : 0 <= r) :
192     IsWeight pentagon (pentagonPathWeight r) := by
193     have hden_pos : 0 < 2 + r := by linarith

```

```

191 constructor
192   · intro a ha
193     cases a <;> dsimp [pentagonPathWeight]
194     · exact div_nonneg zero_le_one hden_pos.le
195     · exact div_nonneg zero_le_one hden_pos.le
196     · exact div_nonneg zero_le_one hden_pos.le
197     · exact div_nonneg zero_le_one hden_pos.le
198     · exact div_nonneg zero_le_one hden_pos.le
199     · exact div_nonneg hr hden_pos.le
200     · exact div_nonneg hr hden_pos.le
201     · exact div_nonneg hr hden_pos.le
202     · exact div_nonneg hr hden_pos.le
203     · exact div_nonneg hr hden_pos.le
204   · intro C hC
205     simp [pentagon, pentagonContexts] at hC
206     rcases hC with hC | hC | hC | hC | hC
207     · subst C
208       simp [pentagonC0, pentagonPathWeight]
209       field_simp [ne_of_gt hden_pos]
210       ring_nf
211     · subst C
212       simp [pentagonC1, pentagonPathWeight]
213       field_simp [ne_of_gt hden_pos]
214       ring_nf
215     · subst C
216       simp [pentagonC2, pentagonPathWeight]
217       field_simp [ne_of_gt hden_pos]
218       ring_nf
219     · subst C
220       simp [pentagonC3, pentagonPathWeight]
221       field_simp [ne_of_gt hden_pos]
222       ring_nf
223     · subst C
224       simp [pentagonC4, pentagonPathWeight]
225       field_simp [ne_of_gt hden_pos]
226       ring_nf
227
228 theorem pentagon_pathWeight_cyclicSum (r : Real) :
229   pentagonCyclicSum (pentagonPathWeight r) = 5 / (2 + r) := by
230   simp [pentagonCyclicSum, pentagonCyclicAtoms, pentagonPathWeight]
231   ring_nf
232
233 theorem pentagon_path_exceeds_classical_iff {r : Real} (hr : 0 <= r) :
234   2 < pentagonCyclicSum (pentagonPathWeight r) <-> r < 1 / 2 := by
235   have hden_pos : 0 < 2 + r := by linarith
236   rw [pentagon_pathWeight_cyclicSum]
237   constructor
238   · intro h
239     field_simp [ne_of_gt hden_pos] at h
240     linarith

```

```

241   · intro h
242     field_simp [ne_of_gt hden_pos]
243     linarith
244
245 theorem pentagon_path_exceeds_kcbs_of_small {r : Real}
246   (hr : 0 ≤ r) (hsmall : r < Real.sqrt 5 - 2) :
247     Real.sqrt 5 < pentagonCyclicSum (pentagonPathWeight r) := by
248   have hden_pos : 0 < 2 + r := by linarith
249   have hsqrt_pos : 0 < Real.sqrt 5 := Real.sqrt_pos.2 (by norm_num)
250   have hlt : 2 + r < Real.sqrt 5 := by linarith
251   have hmul : Real.sqrt 5 * (2 + r) < Real.sqrt 5 * Real.sqrt 5 :=
252     mul_lt_mul_of_pos_left hlt hsqrt_pos
253   have hsq : Real.sqrt 5 * Real.sqrt 5 = 5 := by
254     rw ← [pow_two, Real.sq_sqrt]
255     norm_num
256   rw [pentagon_pathWeight_cyclicSum]
257   field_simp [ne_of_gt hden_pos]
258   nlinarith
259
260 theorem pentagon_pathWeight_strictlyPositive {r : Real} (hr : 0 < r) :
261   StrictlyPositive pentagon (pentagonPathWeight r) := by
262   intro a ha
263   have hden_pos : 0 < 2 + r := by linarith
264   cases a <|> dsimp [pentagonPathWeight]
265   · exact div_pos zero_lt_one hden_pos
266   · exact div_pos zero_lt_one hden_pos
267   · exact div_pos zero_lt_one hden_pos
268   · exact div_pos zero_lt_one hden_pos
269   · exact div_pos zero_lt_one hden_pos
270   · exact div_pos hr hden_pos
271   · exact div_pos hr hden_pos
272   · exact div_pos hr hden_pos
273   · exact div_pos hr hden_pos
274   · exact div_pos hr hden_pos
275
276 theorem pentagon_halfWeight_not_strictlyPositive :
277   ¬ StrictlyPositive pentagon pentagonHalfWeight := by
278   intro hpos
279   have hx := hpos x0 (by simp [pentagon])
280   norm_num [pentagonHalfWeight] at hx
281
282 theorem pentagon_pathWeight_zero_eq_halfWeight :
283   pentagonPathWeight 0 = pentagonHalfWeight := by
284   funext a
285   cases a <|> norm_num [pentagonPathWeight, pentagonHalfWeight]
286
287 end SoftmaxVerification

```